

# Evidence-Bound Recovery for Ambiguous Submissions in Local Generative-AI Pipelines

Tool Description

Anonymous submission draft

**Abstract**—A lost submission response can leave a generation client uncertain whether its backend executed a request. We describe evidence-bound recovery tooling for local image-generation pipelines: a versioned evidence exporter, a same-run submission guard, and controlled fault injection with backend observation. The exporter separates reported state, interpretation and oracle evidence. The guard preserves explicitly unknown outcomes without claiming automatic reconciliation. Four fixed Windows/ComfyUI control and response-loss cases were oracle-evaluable. Each control had one submission and one observed execution; each injected case had an unresolved first call and a resolved second call, with two submissions and executions. The second call created a fresh run with a different seed after observed completion. The demonstration establishes bounded execution visibility, not recovery of the original run or fewer executions. Deployment rates and output-quality improvement remain unmeasured.

## I. INTRODUCTION

Consider a client that submits an image-generation workflow and loses the response after the backend accepts it. The backend may continue while the client lacks the returned job identifier. A second submission creates another opportunity for execution, but a timeout or a count of HTTP requests cannot establish what the backend actually did. The relevant question is which observations justify the next recovery action and the subsequent report of that action.

Uncertain outcomes are an established distributed-systems problem. Remote procedure call designs distinguish call semantics under communication failure [1], while idempotent API designs use caller-provided identifiers and explicit service behavior to make retries safe [2]. A local workstation does not remove the distinction between request transmission and backend execution. Our tool addresses how that distinction is represented and inspected in a particular local generation pipeline; it does not introduce a new exactly-once protocol.

`local-gpu-imagegen` combines persistent run records, generation orchestration, a ComfyUI adapter and artifact handling. We describe tooling that preserves the client’s evidence, applies versioned interpretation rules, and records backend observations independently of the client’s returned status. Its product guard is deliberately narrower: an explicitly unknown submission blocks further submission through the same unresolved run at the reviewed entry points.

The tool description has three parts: an evidence representation and read-only exporter; a same-run guard whose unresolved state remains visible; and CPU and Windows fault-observation workflows with separate evidence boundaries.

The Windows demonstration tests visibility across the client-backend boundary. It does not establish that the guard reduces execution counts in that protocol. The contribution is an inspectable integration of evidence interpretation, run-scoped control and backend observation. Section 6 compares its scope with idempotent APIs, workflow execution and tracing. An exhaustive nearest-tool review remains open; no first-of-its-kind claim is made.

## II. FAILURE MODEL AND EVIDENCE SEMANTICS

A **product call** is an invocation by the research client. A **run** is the persistent product object created by `local_gpu_start_run`. A **submission** in the Windows results is a proxy-observed `POST /prompt`; a call that fails before that boundary need not produce a submission. An **execution instance** is an observation bound to backend execution events. CPU fixtures instead observe the synthetic worker entry. These units are not interchangeable.

F00 is a normal control. F02 suppresses the first accepted submission response, creating a window in which the client lacks an acknowledged job identifier. F03, covered by the CPU fixtures rather than the Windows campaign reported here, concerns loss of completion information after a job identifier is known. An explicit rejection and a missing response have different evidential meanings. Neither should be silently converted into a backend execution outcome.

The evidence model distinguishes reported state, interpreted state and oracle state. In addition, the Windows research client emits its own call-level labels. Its `resolved` label is derived from a generated product result after checking for artifact hashes. Its `unresolved` label can arise from several client errors. Thus a research-client label alone does not prove that a durable product manifest has a particular recovery state. In F02, the proxy’s accepted-response loss and the oracle binding provide the additional context.

## III. TOOL DESIGN

### A. Evidence-preserving export

The exporter accepts a record or run manifest and produces a separate normalized record. It copies reported state, records the mapping version and reasons, and retains supplied oracle state separately. The schema includes source identity, request and input digests, backend identity, job identifier, artifact

hash and validator version. Missing fields carry explicit reasons instead of fabricated values. The inspected mapping is `research-normalization-v2`.

Interpretation checks more than an execution-success label. Job and artifact bindings, approval and validation evidence, and recovery state constrain the `execution_verified` decision. An unresolved recovery state prevents this flag from becoming true even when other evidence reports successful execution. This permits the record to express backend completion without asserting that the product’s recovery obligation has been discharged. The exporter does not invoke the engine, retry a request or update the source run. Its output writer uses exclusive file creation. These are implementation properties, not a claim of complete filesystem security.

### B. Recovery guard and its scope

`AssetRunEngine` and `RunStore` preserve the explicit transport marker `submission_outcome=unknown` as an unresolved attempt. The reviewed submission entry checks that unresolved attempt before permitting further generation. The guard applies within the same run, including attempts with a different idempotency key. The corresponding inspection action is `get_run`.

This guard does not discover a missing backend job identifier or reconcile an unknown job automatically. A newly created run lies outside the prior run’s guard. Existing recovery for a known job in a two-stage operation is a separate path and is not attributed to this no-job-ID guard. The resulting tradeoff is explicit: the guard withholds another submission through that run while the required evidence is unavailable; it does not guarantee eventual completion.

### C. Controlled observation on Windows

Figure 1 shows the product and research boundaries. The research harness adds a transport shim and loopback fault proxy. The shim routes prompt transport through the proxy while retaining the actual backend endpoint in the product’s identity client. F02 drops the first response only after backend acceptance. This arrangement is a research instrument, not a native ComfyUI recovery feature.

The observer connects before the product invocation and uses the operation key as its ComfyUI client identifier, matching the product’s event-routing identity. `ComfyUIEventOracle` requires an execution-start event, a terminal event and completed history bound to a prompt identifier. It derives an execution-instance identifier using the backend boot identity, prompt identifier and observed start sequence. Its independence is from the product return value: it still trusts the backend’s event and history interfaces, and it is not a physical GPU monitor or a crash-durable audit log.

The controller permits the second F02 product call only after the first execution has a bindable completion. The client creates a new run on each call and uses `seed = 4100 + call_index`. This is observation-gated fresh-run generation, not automatic reconciliation of the original run. Matching frozen request digests does not establish full prompt equality:

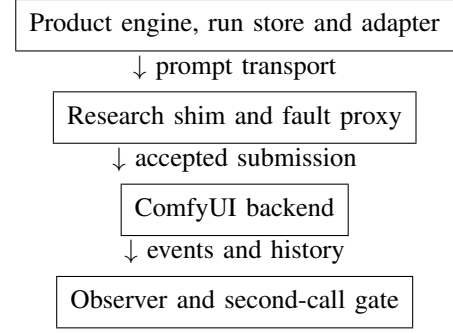


Fig. 1. Product and research boundaries. The controller’s second call creates a fresh run. The exporter separately normalizes reported records; it does not trigger the controller or change the source run.

the inspected digest function excludes the seed and does not use its plan argument.

## IV. EVIDENCE AND DEMONSTRATION

### A. CPU components and deterministic fixtures

The CPU harness separates submission events from synthetic worker-entry events. Its fixtures cover normal operation and response loss before and after job-ID acquisition, including single-stage and two-stage paths. Oracle self-checks exercise cases in which submission and execution counts differ and in which a shared prompt identifier does not imply a single execution. These are fixed semantic checks, not randomly sampled deployments.

Historical command output records 40 research component tests passing in 7.110 seconds using Python unittest discovery, with OK and exit code 0. The preparation report identifies the runtime as Windows Python 3.15.0a8. A later macOS check targeted the transport shim and campaign controller: seven tests passed in 0.003 seconds, followed by successful compilation and whitespace checks. These are distinct stages; the earlier 40-test result is not a full-suite result for the final shim. Neither count is an experimental sample size. Exact immutable source binding of the historical test worktrees remains incomplete.

Earlier paired CPU results are documented separately. They must not be pooled with the Windows cases: the synthetic same-run path and the fresh-run Windows client do not exercise the same recovery scope.

### B. Four controlled Windows cases

We denote the frozen baseline product revision by B2 and the revision with the same-run ambiguity guard by W3. The endpoint-preserving campaign report records both revisions under F00 and F02. All four fixed cases were oracle-evaluable; the two F00 cases were controls and the two F02 cases had confirmed response-loss injection. Table I reports the observations without converting these counts into a success probability.

The campaign totals were six calls, six proxy submissions and six bound execution instances. The F02 second call used a fresh run and changed seed. Counts were read from the

TABLE I  
FOUR FIXED CASES, CHECKED AGAINST RETAINED REPORT FIELDS. S/E  
ARE PROXY SUBMISSIONS/BOUND EXECUTION INSTANCES; U/R ARE  
UNRESOLVED/RESOLVED RESEARCH-CLIENT CALLS. ALL FOUR CASES  
WERE ORACLE-EVALUABLE.

| System | Case | Calls | S/E | Call sequence |
|--------|------|-------|-----|---------------|
| B2     | F00  | 1     | 1/1 | R             |
| W3     | F00  | 1     | 1/1 | R             |
| B2     | F02  | 2     | 2/2 | U → R         |
| W3     | F02  | 2     | 2/2 | U → R         |

retained campaign report; raw events have not been independently reconstructed in this manuscript revision. No rate is estimated.

The report identifies an RTX 5070 Ti Laptop GPU and ComfyUI v0.30.0 at a pinned source revision, with model and workflow hashes checked against the campaign configuration. These are recorded environment facts for this campaign. They do not establish behavior across other models, backend versions or machines.

Both systems produced two observed executions in F02. This demonstrates that the research observer could retain execution evidence despite the first call’s ambiguous outcome. It does not demonstrate an execution-count advantage for W3. Because the second call changed run and seed, the table also does not establish a duplicate execution of an identical generation request.

#### C. Resolution labels require an explicit aggregation rule

A resolved second call does not erase the first call’s unresolved result or show that its original run was reconciled. Read-only inspection of the retained campaign report confirmed that both F02 records carry `resolved=1` and `unresolved=1`. These are independent any-call flags: across the four cases, `resolved` sums to four and `unresolved` to two. They are overlapping counts, not mutually exclusive outcomes. The earlier Markdown summary’s zero unresolved cases is therefore not the controller aggregation. Table I preserves the call sequence; the evidence addendum records this correction without rewriting the historical report. No recovery-success rate is inferred.

There is also a counting distinction within the observer export. Successive oracle decisions retain cumulative raw-event snapshots, so summing their event counts can count earlier events again. The controller’s execution count uses distinct bound execution-instance identifiers. A raw-evidence audit must check those bindings, not substitute aggregate event counts.

#### V. REPRODUCIBILITY AND LIMITATIONS

The source and retained reports support inspection of the protocol. Frozen B2 and W3 product revisions are distinct from the research harness revision; a reproduction must preserve all three identities. Historical commands and source identifiers are recorded in the companion evidence ledger. A fresh clean-checkout reproduction has not been performed for this revision.

Raw Windows campaign JSONL, private configurations, model files and generated images remain Windows-local according to the retained report. Read-only follow-up inspected the retained campaign-report fields and computed its file hash. This did not independently reconstruct raw WebSocket events or verify image content. A sanitized per-call event/history manifest and its binding checks are still required for independent inspection of Table I. The availability of source and a summary is not equivalent to a complete distributable reproduction package.

Artifact evidence has its own boundary. The research client computes content hashes for returned artifacts, whereas oracle-exported path fingerprints hash metadata strings. A path fingerprint is not a content hash. Completed backend history and a content hash do not establish visual quality, task acceptance or finalization. Earlier protected-pixel or mask-validation results from a different pilot are not imported into this campaign.

Observation depends on receiving and binding backend events within the harness window. Missing events limit evaluability; they do not establish non-execution. The same-process CPU observer and backend-origin Windows observer have different failure boundaries, neither of which establishes crash-complete auditability. The fixed case set contains no basis for deployment failure frequency, natural duplicate-execution frequency, performance improvement or broad model robustness. No GPU run, model download or new experiment was performed for this manuscript revision.

#### VI. RELATED WORK AND DISCUSSION

RPC failure semantics provide the conceptual starting point: a failed communication does not necessarily settle whether a remote operation ran [1]. Idempotent API design addresses safe retries through an explicit contract, including caller-provided request identifiers [2]. Our tool relies on neither a prompt identifier nor a client idempotency key as sufficient evidence of backend deduplication. Its role is to preserve and qualify observations when such a guarantee has not been established for the evaluated path.

The practical distinction is between observation and control. Backend events can establish that work completed while the client’s original run remains unresolved. Conversely, a conservative same-run guard can block a further submission without resolving that uncertainty. The Windows harness crosses a different boundary by creating a new run after observation. Keeping these scopes separate explains why the demonstration can be oracle-evaluable without showing automatic recovery or a W3 reduction in executions.

Temporal’s Activity documentation describes the same lost-completion boundary: a worker may finish an Activity and crash before reporting completion, leading to a retry. It recommends idempotent Activities [3]. This is an important comparison because durable workflow state alone does not eliminate uncertainty about external side effects. Our same-run guard withholds a submission under explicit uncertainty, but supplies no equivalent general workflow-execution or automatic reconciliation guarantee.

OpenTelemetry represents operations through spans and uses trace context to relate observations across a request [4]. Such correlation provides a useful representation for a distributed execution path. In this tool, the additional application-specific rule is which observations permit an execution claim: backend start and terminal events must bind to completed history, and unresolved product recovery can still veto `execution_verified`. Correlation identifiers alone are not used as execution counts.

Toxiproxy provides configurable TCP connection fault injection for testing [5]. Our research proxy uses an application-level acceptance condition to select the response to suppress, while the observer separately checks execution. This comparison does not show that Toxiproxy could not support an equivalent setup; no comparative implementation or usability measurement was performed.

This is a documentation-based comparison of responsibilities, not a performance benchmark or a feature-absence survey. The practical contribution is making the relationship between client ambiguity, backend observation and permitted claims inspectable for a local generation workflow. A broader academic and reconciliation-system nearest-work comparison remains incomplete. No superiority or exhaustive novelty claim follows from the selected sources.

## VII. CONCLUSION

The tool preserves ambiguous client outcomes alongside separately bound execution evidence and implements a conservative guard within an unresolved run. The controlled Windows cases demonstrate execution visibility under lost acceptance responses, while exposing the distinction between a new successful call and reconciliation of the original run. These bounded observations support an evidence-preserving tool description. They do not establish automatic unknown-job recovery, deployment-wide reliability or natural duplicate-execution rates.

## REFERENCES

- [1] A. D. Birrell and B. J. Nelson, “Implementing remote procedure calls,” *ACM Transactions on Computer Systems*, vol. 2, no. 1, pp. 39–59, 1984. [Online]. Available: <https://www.cs.cmu.edu/~15712/papers/birrell84.pdf>
- [2] M. Featonby, “Making retries safe with idempotent apis,” Amazon Builders’ Library, accessed September 27, 2026. [Online]. Available: <https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/>
- [3] Temporal Technologies, “Activity definition,” official documentation. Accessed September 27, 2026. [Online]. Available: <https://docs.temporal.io/activity-definition>
- [4] OpenTelemetry Authors, “Traces,” official documentation. Accessed September 27, 2026. [Online]. Available: <https://opentelemetry.io/docs/concepts/signals/traces/>
- [5] Shopify, “Toxiproxy,” official project documentation. Accessed September 27, 2026. [Online]. Available: <https://github.com/Shopify/toxiproxy>